



CLASS TEST

COMS3005A – Advanced Analysis of Algorithms

Semester 1 2023

54 Marks

Duration 90 minutes

This paper consists of 12 pages.

Instructions

1. Make sure you write your name and student number on the paper.
2. Answer on the question paper.
3. Answer all questions.
4. The mark for each question is given in brackets next to each question.
5. Read each question carefully before answering: always make sure you provide an answer to what is being asked.

Surname: _____

First name: _____

Student number: _____

[TURN PAGE]

Question 1

Computability

[6]

- (a) *Turing Machines (TMs)* are more powerful than *Pushdown Automata (PDAs)* and *PDAs* are more powerful than *Finite Automata (FAs)*.

Explain why this is the case.

(3)

- (b) Explain why the *Universal Turing Machine (UTM)* is an important mathematical model in computability theory.

(3)

Question 2

Complexity

[11]

- (a) Suppose you analyse the runtime of a given algorithm and you come up with the complexity function below:

$$f(n) = 13n^2 + 15n + 345$$

What can you say about the order of complexity of the algorithm?

Justify your answers.

(4)

- (b) Consider the algorithm given below which will return the smallest number in the list of numbers.

```
00  Algorithm myMin(myList, n)
    Input:  myList, n where myList is an array with n entries
            (indexed 0...n - 1)
    Output: myList[n - 1] which contains the smallest number in the list
01      i ← n - 1
02      For j from 0 to i - 1
03          If myList[j] < myList[j + 1]
04              Then
05                  Swop(myList[j], myList[j + 1])
06      Return myList[n - 1]
```

- i. Explain how the algorithm solves the problem.

(1)

- ii. If *swops* are considered to be the *basic operation* of this algorithm, when would the *best* and *worst* cases occur? (2)

- iii. Determine the *average* case performance of the algorithm in terms of the number of *swops* made to find the smallest number in a list of length n . State any assumptions that you make. (4)

Question 3

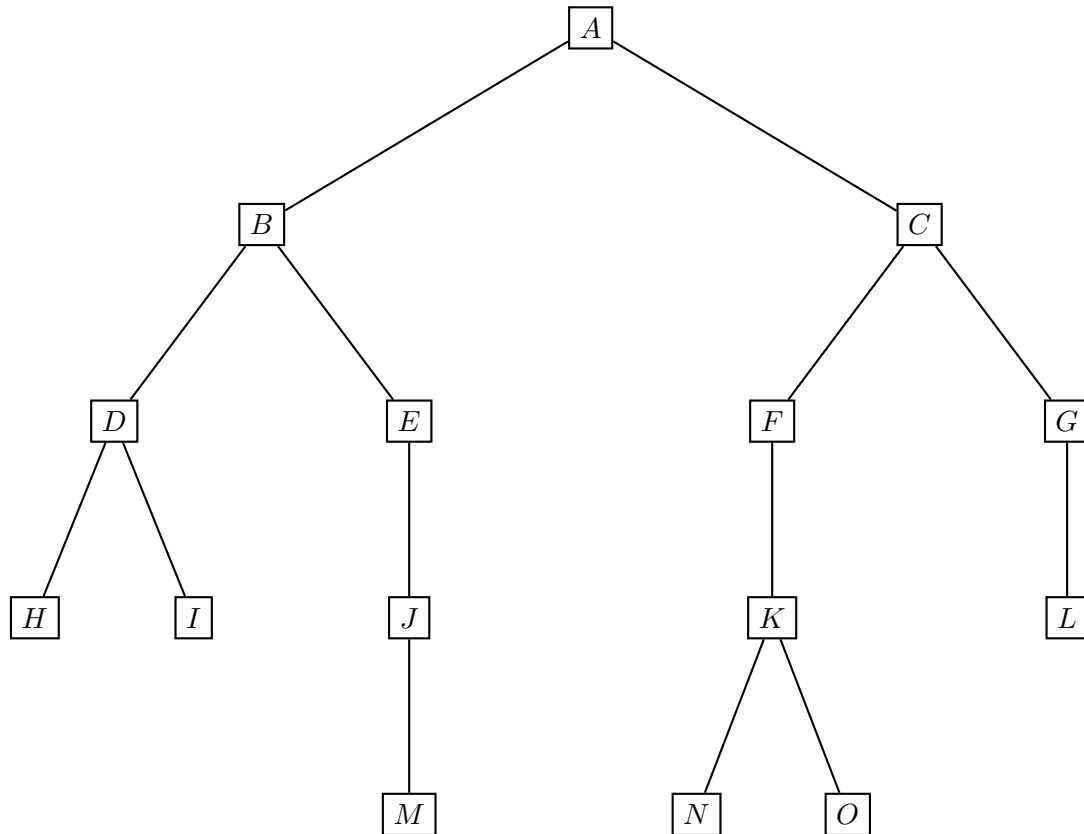
Uninformed Search

[4]

(a) Consider the tree shown below.

Show how **depth first search** would search the tree to find the node labeled *G*.

Be sure to show which node is being explored, which nodes have already been explored and which nodes are in the frontier.
(4)



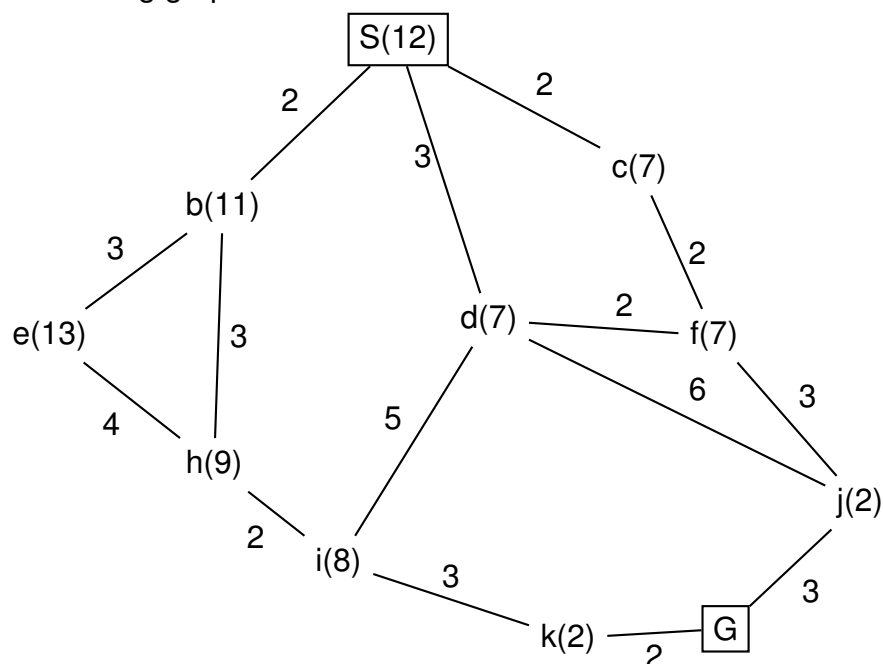
[TURN PAGE]

Question 4

Informed Search

[17]

(a) Consider the following graph:



Each node shows the $\hat{h}$ value (estimate to the goal) in parentheses after the node name. Each edge shows the $\hat{g}$ value next to it.

Assume that an A* search is employed, that the start node is S , and the goal is G .

The first 2 steps (start node added to frontier, the expansion of that node, and the state of the frontier) are shown in the table below. The *node expanded* column shows the $\hat{g}$, $\hat{h}$, and $\hat{f}$ values in parentheses after the node name, and the frontier shows the $\hat{f}$ value in parentheses after the front-most node in the path has been explored):

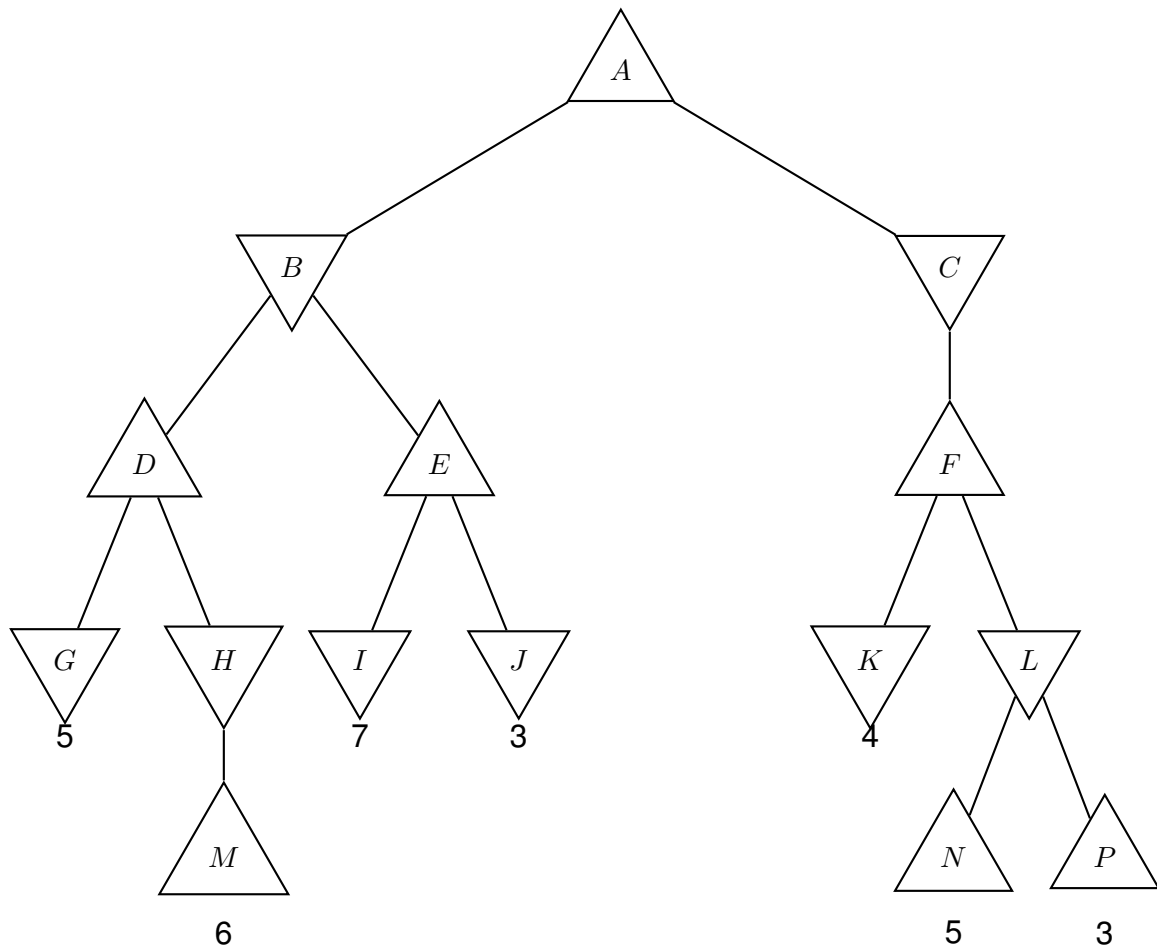
Step	Node expanded	Frontier
1		[< $S(12)$ >]
2	$S(0, 12, 12)$	[< $c(9), S$ >, < $d(10), S$ >, < $b(13), S$ >]

Complete the table by showing the list of nodes in the order in which they are expanded (not just explored) and the state of the open queue when the nodes are expanded.

(8)

(b) Consider the game tree below and then answer the questions that follow.

Note: A is a *max* node and the static utility values for the leaf nodes are provided below each leaf node.



- i. Suppose a Minimax search is performed on the tree. Provide the min-max values for nodes A, B, C, D, E and F . (3)

- [illegible]

Question 5

Classical Search

[6]

- (a) Suppose you are running the version of the linear search that handles the situation where the *key* may or may not be in the given input list.

In order to empirically determine the average case running time of the algorithm, you repeatedly generate a random number and search for this in a list of a given length. You record the runtime for each search and then average these times and record that time as the average runtime for a list of that input size.

Explain why this approach to determining the average runtime could give you a biased measurement of the average case. (3)

- (b) Suppose that you are running the bisection search algorithm on a list of numbers that contains a number of repeats of the key you are searching for.

Could this affect the number of comparisons made to find the key? Explain your answer. (3)

Question 6	Sorting	[10]
------------	---------	-------------

(a) Trace the execution of *mergesort* on the list of numbers given below.

23, 45, 13, 19, 17, 34, 26, 55

(5)

- [illegible]